

RELATÓRIO 4 - BUSCA SEQUENCIAL, MERGESORT E BUSCA BINÁRIA

Inserção de registros sem duplicidade em arquivo CSV

Disciplina: Análise de Algoritmos - Ciência da Computação - UCB

Grupo 5

Repositório: <https://github.com/branchescunha/algorithm-analysis/tree/main/report-04-search-and-sort>

1. Introdução

O trabalho implementa e compara duas soluções para inserir registros em um CSV sem duplicar uma chave de unicidade. A Solução 1 verifica duplicidade por busca sequencial, nas formas iterativa e recursiva. A Solução 2 mantém em RAM um índice ordenado por MergeSort e usa busca binária, também iterativa e recursiva. A análise foi construída para demonstrar formalmente as complexidades temporal e espacial e confrontá-las com medições experimentais explicitando todas as hipóteses usadas.

2. Base de dados e chave de unicidade

Foi utilizado `cnes_estabelecimentos.csv`, extraído do Cadastro Nacional de Estabelecimentos de Saúde (CNES) no Portal de Dados Abertos. A base possui 636.647 registros e 36 colunas. A validação foi executada sobre o arquivo integral: o script `reduzir.py` lê `CO_UNIDADE` como texto, verifica valores ausentes com `isna()`, unicidade com `is_unique` e ausência de ordenação crescente/decrescente com `is_monotonic_increasing` e `is_monotonic_decreasing`. O resultado foi 636.647 valores distintos e ausência de ordenação no arquivo original. Por isso, `CO_UNIDADE` atende simultaneamente aos dois requisitos do enunciado.

`CO_UNIDADE` é tratada como texto porque existem valores alfanuméricos. Em C, a igualdade e a ordem lexicográfica são avaliadas com `strcmp`. Como o comprimento máximo da chave é limitado e não cresce com N , o custo de uma comparação de chave é considerado constante na análise assintótica em função do número de registros.

O QUE	COMO	POR QUÊ
Base original	Leitura do CSV CNES em latin1, separado por ponto e vírgula.	Usar uma base pública real, como exigido.
Chave <code>CO_UNIDADE</code>	Verificação no arquivo integral com <code>isna()</code> , <code>is_unique</code> e monotonicidade crescente/decrescente.	Comprovar, de forma reproduzível, que a chave é única e não está ordenada.
Base experimental	Amostra reproduzível de $N=10.000$ registros, <code>random_state=42</code> , salva em UTF-8.	Amplia a escala dos experimentos, melhora a observação dos tempos e mantém os testes reproduzíveis.

3. Hipóteses e desenho experimental

Para evitar ambiguidades entre teoria e experimento, as variáveis usadas são: N = quantidade de registros já existentes no destino; K = quantidade de registros recebidos no arquivo de entrada; D = quantidade de registros de entrada cuja chave já existe; I = quantidade de registros realmente novos. Nos testes principais, N inicia em 10.000 e $D=I=K/2$. Foram avaliados $K = 10, 40, 100, 300, 500, 1000, 2500$ e 5000.

Variável/condição	Valor nos testes	Como é garantido
N inicial	10.000	<code>destino_backup.csv</code> é restaurado antes de cada execução.
K	10, 40, 100, 300, 500, 1000, 2500, 5000	Oito arquivos de entrada gerados por <code>gerar.py</code> .
D	$K/2$	Metade dos registros vem da própria base.
I	$K/2$	Metade vem de fora da amostra de 10.000 registros.
Ordem do teste	Embaralhada	<code>sample(..., random_state=100+K)</code> .
Repetições de tempo	20	<code>benchmark.py</code> executa cada cenário 20 vezes e calcula a média.

A contagem de comparações é a principal evidência lógica, pois é determinística. O contador `long long` é incrementado exatamente quando `CO_UNIDADE` do vetor é comparada ao alvo. Os tempos são medidos com `clock()`; início e fim

envolvem somente a operação analisada e a diferença é dividida por CLOCKS_PER_SEC. O arquivo físico é restaurado antes de cada repetição para impedir contaminação entre testes. Para a versão final, estatisticas.csv, comparacao_teorica.csv, crescimento_n.csv, as tabelas e os gráficos deste relatório correspondem à mesma execução experimental final; após a coleta, os resultados foram congelados para manter rastreabilidade entre dados, figuras e texto.

Nota de notação: nas demonstrações, $T(N)$ representa o custo temporal em função do tamanho N da entrada analisada e $S(N)$ representa o espaço auxiliar. Para manter a notação uniforme, as expressões finais de custo temporal são escritas sempre como $T(N)$. Quando o experimento utiliza parâmetros adicionais, eles são definidos separadamente no texto, sem aparecerem como argumentos de T .

4. Solução 1 - Busca sequencial

4.1 Busca sequencial iterativa - demonstração

No pior caso, a chave procurada não existe no vetor. Portanto, o laço percorre todas as N posições. Considere b como o custo fixo de inicialização/retorno e c como o custo constante de uma iteração.

Passo 1 - escrever o custo como uma soma:

$$T(N) = b + c + c + c + \dots + c \quad (N \text{ parcelas de custo } c)$$

Passo 2 - representar a repetição por somatório:

$$T(N) = b + \sum_{i=1}^N c$$

Passo 3 - resolver o somatório. Como c é constante, somar c N vezes produz Nc :

$$\sum_{i=1}^N c = cN$$

$$T(N) = b + cN$$

Passo 4 - identificar o termo dominante. Dividindo por N :

$$T(N)/N = c + b/N$$

Quando N cresce, b/N tende a zero e resta a constante positiva c . Portanto, $T(N)$ cresce proporcionalmente a N :

$$T(N) = \Theta(N)$$

Demonstração por indução da forma $T(N)=b+cN$

Caso base ($N=0$):

$$T(0) = b$$

$$b + c \cdot 0 = b$$

Logo, a fórmula é válida para $N=0$.

Hipótese de indução: suponha que a fórmula seja válida para $N=k$:

$$T(k) = b + ck$$

Passo indutivo: para uma entrada de tamanho $k+1$ existe exatamente uma iteração a mais, de custo c :

$$T(k+1) = T(k) + c$$

$$T(k+1) = (b + ck) + c$$

$$T(k+1) = b + c(k+1)$$

Conclusão: a propriedade vale para $k+1$. Por indução, $T(N)=b+cN$ para todo $N \geq 0$ e, consequentemente, a busca sequencial iterativa possui complexidade temporal $\Theta(N)$.

4.2 Busca sequencial recursiva - demonstração

Na versão recursiva, cada chamada compara uma posição e reduz o problema de N elementos para $N-1$. No pior caso, a chave não é encontrada. Considere $T(0)=b$ e custo constante c por chamada.

Passo 1 - escrever a recorrência:

$$T(N) = T(N-1) + c$$

Passo 2 - substituir a recorrência repetidamente:

$$T(N) = T(N-2) + 2c$$

$$T(N) = T(N-3) + 3c$$

...

$$T(N) = T(N-k) + kc$$

Passo 3 - atingir o caso base. O caso base ocorre quando $N-k=0$, portanto $k=N$:

$$T(N) = T(0) + Nc$$

$$T(N) = b + cN$$

$$T(N) = \Theta(N)$$

Demonstração por indução

Caso base ($N=0$):

$$T(0)=b = b+c \cdot 0$$

Hipótese de indução: suponha válida $T(k)=b+ck$.

Passo indutivo: aplicando a recorrência para $k+1$:

$$\begin{aligned}T(k+1) &= T(k) + c \\T(k+1) &= b + ck + c \\T(k+1) &= b + c(k+1)\end{aligned}$$

Conclusão: a fórmula é válida para $k+1$; logo, por indução, $T(N)=b+cN=\Theta(N)$. Iterativa e recursiva têm o mesmo crescimento temporal, embora a recursiva utilize mais pilha, como será demonstrado no tópico 7.

4.3 Derivação detalhada do custo no experimento usando $T(N)$

Para manter a notação temporal padrão $T(N)$, nesta subseção N representa a quantidade de registros recebidos no lote e M representa a quantidade de registros existentes inicialmente no destino. Essa mudança é apenas notacional: M corresponde ao N da metodologia experimental e N corresponde ao K usado nas tabelas e gráficos.

$$q = N/2$$

Assim, existem q registros duplicados e q registros novos. O custo temporal total do lote será representado exclusivamente por $T(N)$.

Custo dos q registros duplicados

O script gerar.py escolhe como duplicados os primeiros q registros da base. Por isso, o primeiro duplicado é encontrado na primeira posição, o segundo na segunda, e assim por diante.

Passo 1 - escrever a soma das comparações dos duplicados:

$$1 + 2 + 3 + \dots + q$$

Passo 2 - usar a soma dos q primeiros naturais:

$$q(q+1)/2$$

Custo dos q registros novos

Uma chave nova não é encontrada e, portanto, percorre toda a base. Após cada inserção, a base em memória cresce uma posição. Os tamanhos percorridos são $M, M+1, \dots, M+q-1$.

Passo 1 - escrever a soma:

$$M + (M+1) + (M+2) + \dots + (M+q-1)$$

Passo 2 - separar a parcela N da parcela variável:

$$qM + [0 + 1 + 2 + \dots + (q-1)]$$

Passo 3 - resolver o somatório dos inteiros de 0 até $q-1$:

$$\begin{aligned}0 + 1 + \dots + (q-1) &= q(q-1)/2 \\qM + q(q-1)/2\end{aligned}$$

Custo total do lote

Passo 1 - somar os custos dos duplicados e dos novos:

$$\begin{aligned}T(N) &= q(q+1)/2 + qM + q(q-1)/2 \\T(N) &= qM + [q(q+1) + q(q-1)]/2\end{aligned}$$

Passo 2 - desenvolver e simplificar as parcelas:

$$\begin{aligned}T(N) &= qM + [q^2 + q + q^2 - q]/2 \\T(N) &= qM + 2q^2/2 \\T(N) &= qM + q^2\end{aligned}$$

Passo 3 - substituir $q=N/2$:

$$T(N) = (N/2)M + (N/2)^2$$

$$T(N) = (N/2)M + N^2/4$$

Essa é a equação de custo do workload controlado: o primeiro termo representa as comparações causadas pelo tamanho inicial da base e o segundo representa o crescimento provocado pelas inserções do próprio lote.

Substituição de valores experimentais com $M=10.000$

$$\begin{aligned}T(10) &= (10/2) \cdot 10000 + 10^2/4 = 50000 + 25 = 50025 \\T(100) &= (100/2) \cdot 10000 + 100^2/4 = 500000 + 2500 = 502500\end{aligned}$$

$$T(1000) = (1000/2) \cdot 10000 + 1000^2/4 = 5000000 + 250000 = 5250000$$

$$T(5000) = (5000/2) \cdot 10000 + 5000^2/4 = 25000000 + 6250000 = 31250000$$

Os resultados apresentados na tabela experimental coincidem exatamente com as contagens observadas. Assim, nesta demonstração o custo temporal é expresso somente por $T(N)$, como função do tamanho do lote, mantendo $M=10.000$ fixo. No restante do relatório, as tabelas preservam a nomenclatura experimental original (N para a base e K para o lote). Uma única busca sequencial continua tendo complexidade $\Theta(N)$ em função do tamanho da estrutura pesquisada.

5. Solução 2 - índice em RAM, MergeSort e busca binária

O CSV no disco não é ordenado. O programa carrega apenas CO_UNIDADE para um vetor em RAM, ordena esse índice e faz as consultas sobre ele. Um registro aceito é escrito no fim do CSV com append; somente a chave é inserida na posição correta do vetor em RAM. Assim, a exigência de manter o arquivo físico desordenado é preservada enquanto a busca binária recebe uma estrutura ordenada.

5.1 MergeSort recursivo - demonstração temporal

O MergeSort recursivo divide o vetor em duas metades e depois intercala os resultados. A intercalação percorre linearmente os elementos envolvidos, produzindo a recorrência:

$$T(N) = 2T(N/2) + cN$$

Passo 1 - primeira expansão:

$$T(N) = 2[2T(N/4) + c(N/2)] + cN$$

$$T(N) = 4T(N/4) + 2cN$$

Passo 2 - segunda expansão:

$$T(N) = 8T(N/8) + 3cN$$

Passo 3 - generalizar após j expansões:

$$T(N) = 2^j T(N/2^j) + jcN$$

Passo 4 - determinar quando a divisão termina:

$$N/2^j = 1$$

$$N = 2^j$$

$$j = \log_2 N$$

Passo 5 - substituir $j=\log_2 N$ na expressão geral:

$$T(N) = 2^{(\log_2 N)} T(1) + cN \log_2 N$$

$$T(N) = NT(1) + cN \log_2 N$$

O termo $N \log N$ cresce mais rapidamente do que N . Logo:

$$T(N) = \Theta(N \log N)$$

A estrutura de divisões é a mesma no melhor, médio e pior caso; por isso a ordem assintótica permanece $\Theta(N \log N)$. A dedução espacial completa é apresentada no tópico 7.

5.2 MergeSort iterativo - demonstração temporal

Na versão iterativa, os blocos a serem intercalados dobram de tamanho a cada passagem: 1, 2, 4, 8, ..., até cobrir N elementos.

Passo 1 - após p passagens, o tamanho do bloco é 2^p . Para cobrir todo o vetor:

$$2^p \geq N$$

$$p \geq \log_2 N$$

$$p = \lceil \log_2 N \rceil$$

Passo 2 - em cada passagem, todos os N elementos participam de alguma intercalação. Se o custo por elemento é c :

$$\text{custo por passagem} = cN$$

Passo 3 - multiplicar o custo de uma passagem pelo número de passagens:

$$T(N) = cN \cdot \lceil \log_2 N \rceil + b$$

Ignorando constantes e o teto, que altera o resultado em no máximo uma passagem:

$$T(N) = \Theta(N \log N)$$

A versão iterativa evita a pilha recursiva, mas ainda usa os buffers temporários da intercalação. O cálculo espacial completo aparece no tópico 7.

5.3 Busca binária iterativa e recursiva - demonstração

A busca binária trabalha sobre o vetor ordenado. A cada comparação, no máximo metade do intervalo continua sendo candidata. Se o intervalo inicial possui N elementos, após k reduções restam no máximo $N/2^k$ elementos.

Cálculo algébrico do número de reduções

Passo 1 - exigir que reste no máximo um elemento:

$$N/2^k \leq 1$$

Passo 2 - multiplicar os dois lados por 2^k :

$$N \leq 2^k$$

Passo 3 - aplicar logaritmo de base 2:

$$\log_2 N \leq k$$

Portanto, o número de reduções é proporcional a $\lceil \log_2 N \rceil$ e a busca binária executa $\Theta(\log N)$ comparações no pior caso.

Expansão algébrica da versão recursiva

$$T(N) = T(N/2) + c$$

$$T(N) = T(N/4) + 2c$$

$$T(N) = T(N/8) + 3c$$

...

$$T(N) = T(N/2^k) + kc$$

Quando $N/2^k=1$, temos $k=\log_2 N$:

$$T(N) = T(1) + c \log_2 N$$

$$T(N) = \Theta(\log N)$$

Demonstração por indução ($N=2^m$)

Caso base ($m=0$, $N=1$):

$$T(1) = b = b + c \cdot 0$$

Hipótese de indução: suponha válida a expressão para $N=2^m$:

$$T(2^m) = b + mc$$

Passo indutivo: para $N=2^{(m+1)}$:

$$T(2^{(m+1)}) = T(2^m) + c$$

$$T(2^{(m+1)}) = b + mc + c$$

$$T(2^{(m+1)}) = b + (m+1)c$$

Conclusão: por indução, $T(N)=b+c \log_2 N$ para potências de 2, portanto $T(N)=\Theta(\log N)$. Para outros N , pisos e tetos alteram apenas uma quantidade constante de passos e não mudam a classe assintótica. A versão iterativa faz as mesmas reduções, mas sem empilhar chamadas.

5.4 Custo global - decomposição passo a passo e comparação das soluções

Além de analisar cada algoritmo isoladamente, é necessário decompor o custo das soluções completas.

Solução 1 - custo global no pior caso

Para manter a mesma convenção de notação, nesta subseção M representa o tamanho inicial da base e N representa a quantidade de registros do lote. Cada busca sequencial pode percorrer uma base que começa com M elementos e cresce com as inserções.

Passo 1 - considerar N buscas sobre uma base de tamanho no máximo $M+N$:

$$T_1(N) \leq N(M+N)$$

Passo 2 - distribuir N :

$$T_1(N) \leq NM + N^2$$

$$T_1(N) = O(NM + N^2)$$

No workload controlado da seção 4.3 foi possível obter uma fórmula exata mais específica porque metade do lote é duplicada, metade é nova e a ordem das duplicatas é conhecida.

Solução 2 - custo global no pior caso

Parcela A - ordenação inicial por MergeSort da base com M elementos:

$$T_{ord}(M) = \Theta(M \log M)$$

Parcela B - N buscas binárias. Como o índice pode crescer até $M+I$:

$$T_{busca}(N) \leq N \cdot c \log_2(M+I)$$

$$T_{busca}(N) = O(N \log(M+I))$$

Parcela C - manutenção do vetor ordenado para I inserções. A j -ésima inserção pode deslocar até $M+j$ elementos:

$$T_{manut}(I) \leq M + (M+1) + \dots + (M+I-1)$$

$$T_{\text{manut}}(I) = IM + [0+1+\dots+(I-1)]$$

$$T_{\text{manut}}(I) = IM + I(I-1)/2$$

$$T_{\text{manut}}(I) = O(IM + I^2)$$

Passo final - somar as três parcelas:

$$T_2(N) = T_{\text{ord}}(M) + T_{\text{busca}}(N) + T_{\text{manut}}(I)$$

$$T_2(N) = O(M \log M + N \log(M+I) + IM + I^2)$$

No experimento principal, $M=10.000$ e $I=N/2$, com I variando de 5 até 2500. A busca binária isolada continua $\Theta(\log M)$, mas a solução completa inclui o custo inicial de ordenar e o custo de manter o vetor ordenado após cada inserção. À medida que I cresce, a parcela de manutenção $O(IM+I^2)$ passa a ter peso maior. Essa distinção é essencial para interpretar corretamente os tempos experimentais.

6. Validação experimental da teoria

6.1 Experimento principal: base inicial $N=10.000$ e K variável

K	N_0	D	I	Seq. teoria	Seq. obs.	Bin. obs.	Bin. limite
10	10000	5	5	50025	50025	129	140
40	10000	20	20	200400	200400	512	560
100	10000	50	50	502500	502500	1269	1400
300	10000	150	150	1522500	1522500	3857	4200
500	10000	250	250	2562500	2562500	6444	7000
1000	10000	500	500	5250000	5250000	12857	14000
2500	10000	1250	1250	14062500	14062500	32409	35000
5000	10000	2500	2500	31250000	31250000	65308	70000

A tabela torna explícitas as hipóteses usadas no gráfico: $N_0=10.000$ e $D=I=K/2$. Na busca sequencial, teoria e experimento coincidem exatamente em todos os cenários (diferença zero). Na binária, a coluna “limite” é o máximo de comparações permitido pela redução pela metade em cada consulta, considerando o crescimento do índice após inserções; os valores observados permanecem abaixo desse limite.

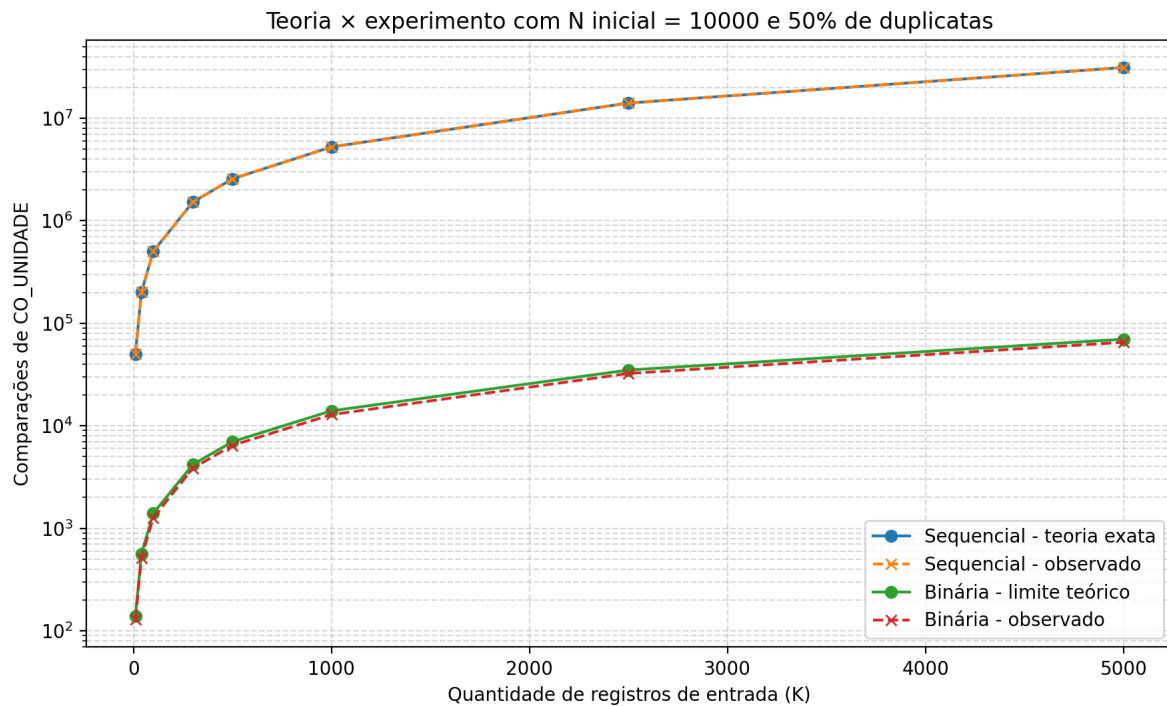


Figura 1 - Teoria × experimento no workload principal ($N_0=10.000$, $D=I=K/2$). A escala logarítmica permite visualizar as diferentes ordens de grandeza.

Na Figura 1, as curvas de teoria e experimento da busca sequencial se sobrepõem. Mantendo $M=10.000$ fixo, para um lote de 5000 registros temos $T(5000)=31.250.000$ tanto na fórmula quanto na medição. A busca binária realiza 65.308 comparações, abaixo do limite teórico de 70.000 para esse cenário. Portanto, o gráfico não é apenas ilustrativo: cada curva está ligada aos valores e hipóteses apresentados na tabela.

6.2 Experimento controlado variando N

Para observar diretamente a dependência em N, foi executado um segundo experimento com uma única chave artificial garantidamente ausente (~~~~ALVO_AUSENTE~~~~) e bases de N=125, 500, 1000, 5000 e 10000. Uma chave ausente força a busca sequencial a percorrer todo o vetor e força a binária a seguir até o intervalo vazio. Foram feitas 50 repetições para os tempos; a quantidade de comparações é determinística.

N	Seq. teoria N	Seq. observada	Bin. teoria $\lfloor \log_2 N \rfloor + 1$	Bin. observada
125	125	125	7	7
500	500	500	9	9
1000	1000	1000	10	10
5000	5000	5000	13	13
10000	10000	10000	14	14

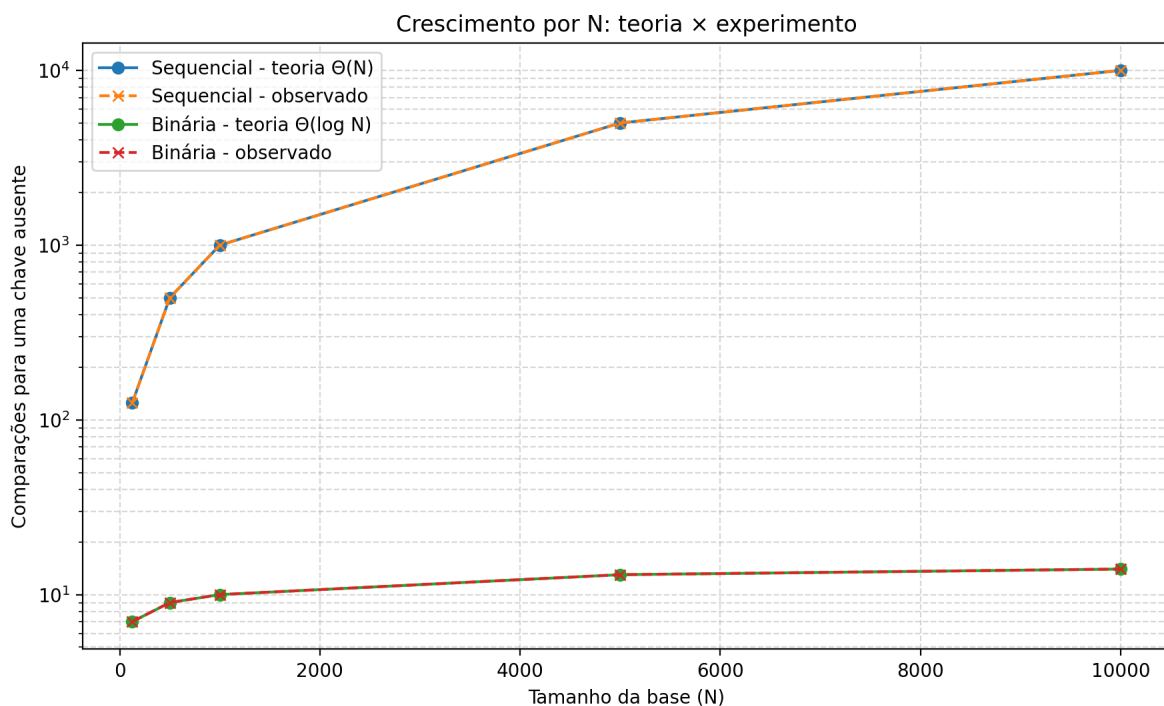


Figura 2 - Crescimento por N com alvo garantidamente ausente. A busca sequencial dobra as comparações quando N dobra; a binária acrescenta aproximadamente uma comparação.

A Figura 2 fornece evidência experimental direta do crescimento: ao aumentar N de 125 para 10.000, a sequencial passa de 125 para 10.000 comparações, enquanto a binária passa apenas de 7 para 14. Os valores observados coincidem com o modelo teórico no experimento controlado, tornando visível a diferença entre crescimento linear e logarítmico.

6.3 MergeSort: teoria x tempo quando N varia

N	MergeSort iterativo (s)	MergeSort recursivo (s)
125	0.000000	0.000000
500	0.000020	0.000160
1000	0.000160	0.000180
5000	0.001360	0.001440
10000	0.003620	0.003640

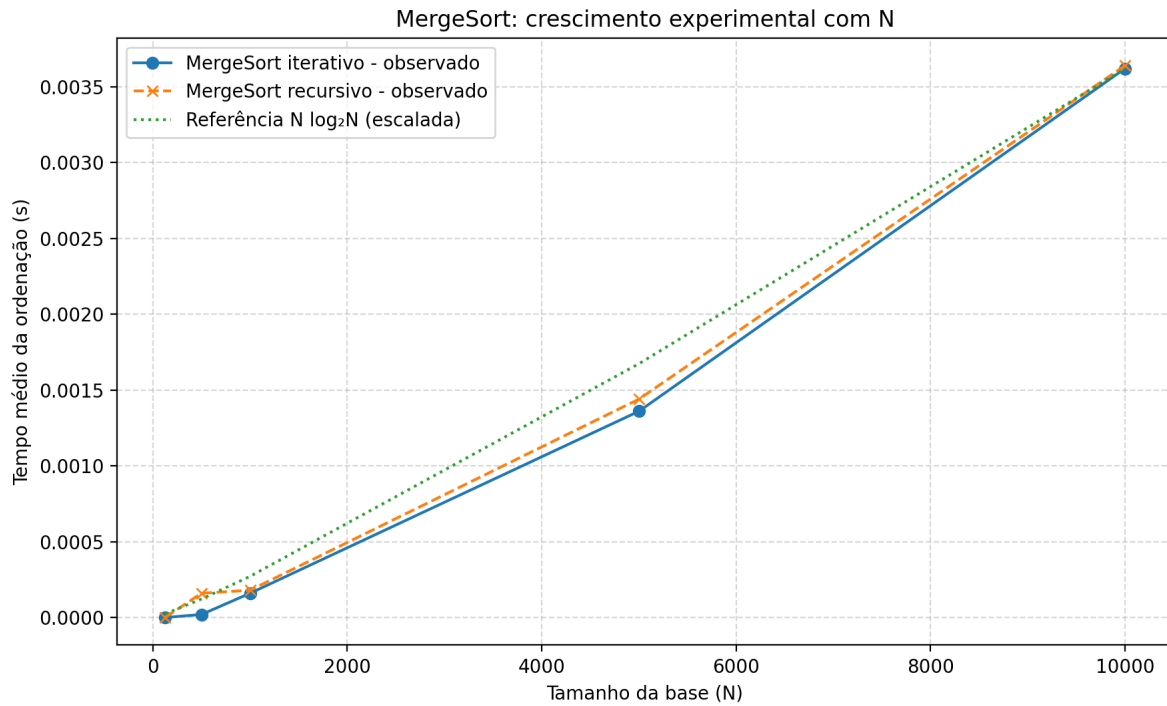


Figura 3 - Tempos médios da ordenação na execução experimental final para N variável; a linha pontilhada é uma referência $N \log_2 N$ escalada.

A Figura 3 é usada como validação de tendência, não como prova matemática. O tempo de ordenação cresce globalmente com N e permanece compatível com a forma $N \log N$ em ordem de grandeza. Para os menores N algumas medições ainda ficam próximas da resolução de clock(); nos maiores tamanhos os tempos tornam-se claramente mensuráveis e crescentes. Por isso, a demonstração formal de $\Theta(N \log N)$ continua sendo a evidência principal e o tempo atua como validação complementar.

6.4 Tempos do workload principal

K	Seq. iter.	Seq. rec.	Sol. 2 iter.	Sol. 2 rec.
10	0.000050	0.000650	0.003750	0.003800
40	0.000800	0.002300	0.003500	0.003900
100	0.001950	0.005950	0.003800	0.003950
300	0.005700	0.017600	0.004550	0.004650
500	0.009500	0.029250	0.005200	0.005250
1000	0.020000	0.059700	0.006800	0.006800
2500	0.061100	0.169800	0.009500	0.009300
5000	0.150850	0.393650	0.016300	0.015050

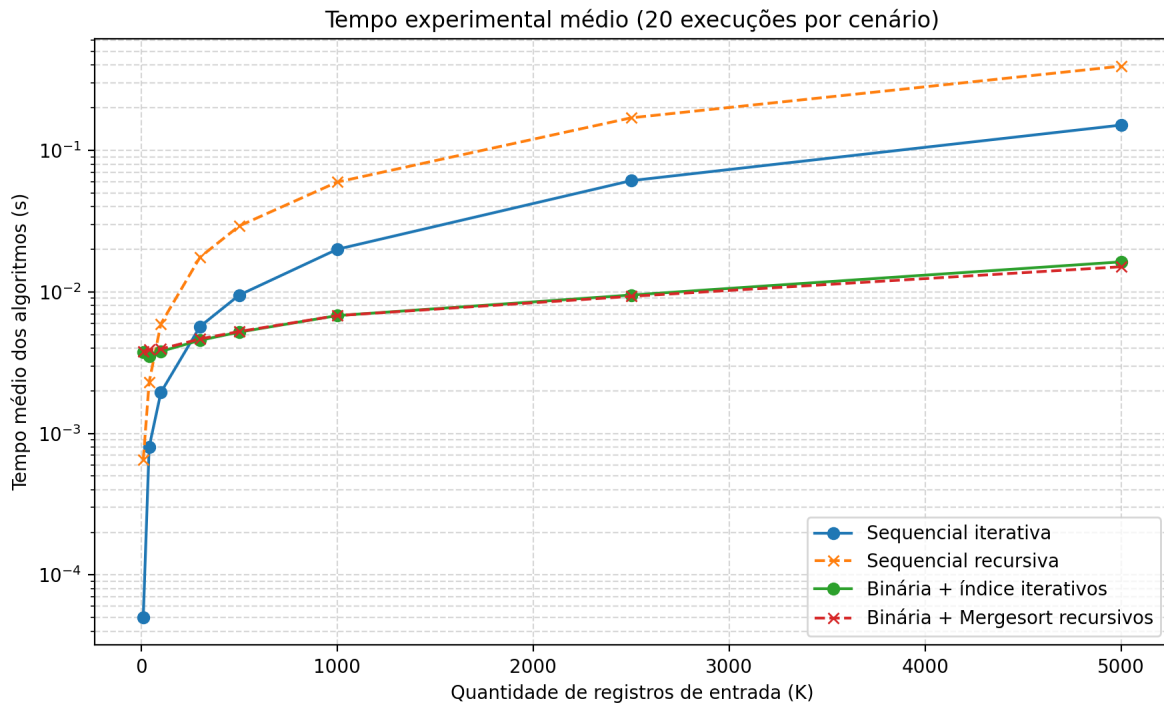


Figura 4 - Tempo médio do workload completo na execução experimental final (20 repetições por K).

A Figura 4 deve ser interpretada junto às complexidades. Com uma base inicial de 10.000 registros, ordenar e manter o índice adiciona um custo fixo/incremental à Solução 2; para lotes pequenos esse overhead pode superar o ganho da busca binária, enquanto para lotes maiores a redução no número de comparações se torna dominante. Isso não contradiz $\Theta(\log N)$: o gráfico mede a solução completa, não somente a função de busca. A correção funcional também foi confirmada: em todos os cenários as quatro implementações inseriram exatamente $I=K/2$ registros novos (5, 20, 50, 150, 250, 500, 1250 e 2500), sem introduzir duplicatas.

7. Demonstração passo a passo e comparação da complexidade espacial

7.1 Busca sequencial iterativa

A versão iterativa não cria novas chamadas de função a cada elemento. Ela reutiliza um número fixo de variáveis: índice i , tamanho N , ponteiros, contador de comparações e algumas variáveis temporárias.

Passo 1 - seja r a quantidade de variáveis auxiliares. r não depende de N .

Passo 2 - se cada uma ocupa no máximo a unidades e existe um custo fixo b :

$$S(N) = a \cdot r + b$$

Passo 3 - como a , r e b são constantes:

$$\begin{aligned} S(N) &= c \\ S(N) &= \Theta(1) \end{aligned}$$

Indução

Caso base: para $N=1$, $S(1)=c$.

Hipótese: suponha $S(k)=c$.

Passo indutivo: ao aumentar a entrada para $k+1$, o laço continua reutilizando as mesmas variáveis e não cria novos frames:

$$S(k+1)=c$$

Conclusão: $S(N)=\Theta(1)$.

7.2 Busca sequencial recursiva

Na versão recursiva, cada chamada fica pendente na pilha enquanto a próxima chamada é executada. No pior caso existem N chamadas ativas antes de atingir o caso base.

Passo 1 - seja f o espaço constante ocupado por um frame e b o espaço do caso base:

$$S(N) = S(N-1) + f$$

Passo 2 - expandir a recorrência:

$$S(N) = S(N-2) + 2f$$

$$S(N) = S(N-3) + 3f$$

...

$$S(N) = S(N-k) + kf$$

Passo 3 - atingir o caso base com $k=N$:

$$S(N) = S(0) + Nf$$

$$S(N) = b + Nf$$

$$S(N) = \Theta(N)$$

Indução

Caso base: $S(0)=b$.

Hipótese: $S(k)=b+kf$.

Passo indutivo:

$$S(k+1)=S(k)+f$$

$$S(k+1)=b+kf+f=b+(k+1)f$$

Conclusão: a pilha cresce linearmente e $S(N)=\Theta(N)$.

7.3 MergeSort iterativo

O MergeSort iterativo não utiliza pilha de recursão. O consumo dominante vem dos vetores temporários criados pela função merge.

Passo 1 - em um merge de m elementos, os dois vetores auxiliares têm tamanhos $n1$ e $n2$:

$$n1 + n2 = m$$

Passo 2 - se cada elemento ocupa a unidades, o espaço dos buffers é:

$$S_{merge}(m) = a(n1+n2) + b$$

$$S_{merge}(m) = am + b$$

Passo 3 - o maior merge ocorre quando $m=N$:

$$S_{merge}(N) = aN + b$$

Os buffers de merges anteriores são liberados antes dos seguintes; portanto seus tamanhos não são somados entre todos os níveis. O pico é linear:

$$S(N) = \Theta(N)$$

7.4 MergeSort recursivo

No MergeSort recursivo existem duas parcelas de memória auxiliar: a pilha de chamadas e os buffers temporários de merge.

Parcela 1 - pilha recursiva

Passo 1 - a profundidade $D(N)$ reduz N pela metade a cada chamada:

$$D(N) = D(N/2) + 1$$

Passo 2 - expandir:

$$D(N) = D(N/4) + 2$$

$$D(N) = D(N/8) + 3$$

...

$$D(N) = D(N/2^k) + k$$

Passo 3 - quando $N/2^k=1$, $k=\log_2 N$:

$$D(N) = \Theta(\log N)$$

Se cada frame ocupa f unidades:

$$S_{pilha}(N) = f \log_2 N + b_1 = \Theta(\log N)$$

Parcela 2 - buffers de merge

No maior merge, os buffers armazenam ao todo N elementos:

$$S_{buffers}(N) = aN + b_2 = \Theta(N)$$

Espaço total

$$S(N) = S_{buffers}(N) + S_{pilha}(N)$$

$$S(N) = aN + f \log_2 N + b$$

Para $N \geq 2$, $\log_2 N \leq N$. Portanto o termo linear domina o termo logarítmico:

$$S(N) = \Theta(N)$$

A recursão adiciona $\Theta(\log N)$ de pilha, mas o pico total continua $\Theta(N)$ por causa dos buffers de intercalação.

7.5 Busca binária iterativa

A busca binária iterativa mantém apenas início, fim, meio, cmp, ponteiros e contador. A quantidade dessas variáveis é fixa e independe do tamanho N .

Passo 1 - seja r essa quantidade constante de variáveis e a o espaço máximo de cada uma:

$$S(N) = a \cdot r + b$$

Passo 2 - como nenhum termo depende de N :

$$S(N) = c$$

$$S(N) = \Theta(1)$$

Indução

Caso base: $S(1)=c$.

Hipótese: $S(k)=c$.

Passo indutivo: aumentar a entrada não cria variáveis adicionais; os mesmos índices são reutilizados:

$$S(k+1)=c$$

Conclusão: $S(N)=\Theta(1)$.

7.6 Busca binária recursiva

Na versão recursiva, cada chamada mantém um frame na pilha e reduz o intervalo pela metade.

Passo 1 - se cada frame ocupa f unidades:

$$S(N) = S(N/2) + f$$

Passo 2 - expandir a recorrência:

$$S(N) = S(N/4) + 2f$$

$$S(N) = S(N/8) + 3f$$

...

$$S(N) = S(N/2^k) + kf$$

Passo 3 - alcançar o caso base quando $N/2^k=1$:

$$k = \log_2 N$$

$$S(N) = S(1) + f \log_2 N$$

$$S(N) = b + f \log_2 N$$

$$S(N) = \Theta(\log N)$$

Indução sobre m , com $N=2^m$

Caso base: $m=0$, $N=1$ e $S(1)=b$.

Hipótese: suponha $S(2^m)=b+mf$.

Passo indutivo:

$$S(2^{(m+1)})=S(2^m)+f$$

$$S(2^{(m+1)})=b+mf+f=b+(m+1)f$$

Conclusão: a profundidade da pilha é logarítmica e $S(N)=\Theta(\log N)$.

7.7 Resumo comparativo

Algoritmo	Tempo	Espaço auxiliar	Demonstração/causa
Busca seq. iterativa	$\Theta(N)$	$\Theta(1)$	N iterações; quantidade constante de variáveis.
Busca seq. recursiva	$\Theta(N)$	$\Theta(N)$	Recorrência $S(N)=S(N-1)+f$ gera N frames.
MergeSort iterativo	$\Theta(N \log N)$	$\Theta(N)$	log N passagens; buffers de merge somam N no pico.
MergeSort recursivo	$\Theta(N \log N)$	$\Theta(N)$	Heap $\Theta(N)$ + pilha $\Theta(\log N)$; domina $\Theta(N)$.
Busca bin. iterativa	$\Theta(\log N)$	$\Theta(1)$	Intervalo é reduzido pela metade; índices constantes.
Busca bin. recursiva	$\Theta(\log N)$	$\Theta(\log N)$	Uma chamada por nível: profundidade log N.

Além do espaço auxiliar específico demonstrado em 7.1 a 7.6, as duas soluções carregam as N chaves em RAM. Esse vetor-base representa $\Theta(N)$ de armazenamento comum ao experimento e não deve ser contado duas vezes ao comparar os algoritmos. A tabela acima resume a memória adicional específica de cada implementação, enquanto as subseções anteriores mostram todos os passos algébricos usados para chegar a cada classe assintótica.

8. Conclusão

CO_UNIDADE atende aos requisitos de unicidade e ausência de ordenação na base original. As quatro implementações impediram a introdução de duplicatas, e a Solução 2 preservou o CSV físico desordenado ao ordenar apenas o índice em RAM.

As demonstrações temporais foram apresentadas por expansão algébrica e por indução: $\Theta(N)$ para busca sequencial, $\Theta(\log N)$ para busca binária e $\Theta(N \log N)$ para MergeSort. A análise espacial também foi derivada passo a passo: a recursão aumenta a pilha para $\Theta(N)$ na sequencial e $\Theta(\log N)$ na binária; no MergeSort, os buffers temporários de $\Theta(N)$ dominam a pilha $\Theta(\log N)$, mantendo o espaço auxiliar total em $\Theta(N)$.

Os experimentos sustentam essas conclusões. No workload com base inicial $N=10.000$, a fórmula exata da sequencial coincidiu com todos os dados observados e a binária permaneceu abaixo de seu limite teórico. No maior lote, a sequencial realizou 31.250.000 comparações, enquanto a binária realizou 65.308. No experimento variando N, a sequencial cresceu de 125 para 10.000 comparações enquanto a binária cresceu de 7 para 14, tornando visível a diferença entre crescimento linear e logarítmico. Os tempos foram tratados como evidência complementar; as contagens lógicas e as demonstrações formais permanecem a base da validação. Em todos os cenários, as quatro implementações produziram a quantidade esperada de inserções válidas, confirmando que a comparação de desempenho foi feita entre soluções funcionalmente equivalentes.

9. Repositório público

O repositório contém os códigos em C, arquivos usados nos testes, scripts de preparação e benchmark, resultados CSV, gráficos e instruções de compilação e execução:

<https://github.com/branchescunha/algorithm-analysis/tree/main/report-04-search-and-sort>